

Logs do sistema



Sistema de Auditoria via Triggers (PostgreSQL)

Este documento contém a estrutura de banco de dados para auditoria nativa no PostgreSQL usando tabelas de log e triggers para os módulos `accounts`, `program` e `collection`.



1. Configuração da Infraestrutura de Auditoria

Criação do schema isolado, tabela central de logs utilizando o formato flexível `JSONB` e a função que processará os gatilhos.

```
-- =====
--
-- 1 - Criar o schema e a tabela de log
-- =====
--
CREATE SCHEMA IF NOT EXISTS audit;

CREATE TABLE IF NOT EXISTS audit.sistema_audit_log (
    id SERIAL PRIMARY KEY,
    table_name VARCHAR(255) NOT NULL,
    action_type VARCHAR(10) NOT NULL CHECK (
        action_type IN ('INSERT', 'UPDATE', 'DELETE')
    ),
    record_id INTEGER,
    old_data JSONB,
    new_data JSONB,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    db_user VARCHAR(255) DEFAULT current_user
);

-- =====
--
```

```

-- 2 – Criar a função de auditoria
-- =====
=

CREATE OR REPLACE FUNCTION audit.audit_trigger_func()
RETURNS TRIGGER AS $$
BEGIN

    IF (TG_OP = 'DELETE') THEN

        INSERT INTO audit.sistema_audit_log (
            table_name,
            action_type,
            record_id,
            old_data
        )
        VALUES (
            TG_TABLE_NAME,
            TG_OP,
            OLD.id,
            row_to_json(OLD)::jsonb
        );

        RETURN OLD;

    ELSIF (TG_OP = 'UPDATE') THEN

        IF ROW(OLD.*) IS DISTINCT FROM ROW(NEW.*) THEN

            INSERT INTO audit.sistema_audit_log (
                table_name,
                action_type,
                record_id,
                old_data,
                new_data
            )
            VALUES (
                TG_TABLE_NAME,

```

```

        TG_OP,
        NEW.id,
        row_to_json(OLD)::jsonb,
        row_to_json(NEW)::jsonb
    );

END IF;

RETURN NEW;

ELSIF (TG_OP = 'INSERT') THEN

    INSERT INTO audit.sistema_audit_log (
        table_name,
        action_type,
        record_id,
        new_data
    )
    VALUES (
        TG_TABLE_NAME,
        TG_OP,
        NEW.id,
        row_to_json(NEW)::jsonb
    );

    RETURN NEW;

END IF;

RETURN NULL;

END;
$$ LANGUAGE plpgsql;

```

2. Vínculo dos Triggers nas Tabelas do Sistema

⚠ **Atenção:** Certifique-se de aplicar os scripts abaixo após as tabelas originais do Django já terem sido criadas no banco de dados.

App: accounts

```
-- Usuários Base
DROP TRIGGER IF EXISTS audit_usuario_trigger ON accounts_usuario;
CREATE TRIGGER audit_usuario_trigger
AFTER INSERT OR UPDATE OR DELETE ON accounts_usuario
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Roles
DROP TRIGGER IF EXISTS audit_role_trigger ON accounts_role;
CREATE TRIGGER audit_role_trigger
AFTER INSERT OR UPDATE OR DELETE ON accounts_role
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Vínculo N:N entre Usuários e Roles
DROP TRIGGER IF EXISTS audit_usuario_roles_trigger ON accounts_usuario_roles;
CREATE TRIGGER audit_usuario_roles_trigger
AFTER INSERT OR UPDATE OR DELETE ON accounts_usuario_roles
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```

App: program

```
-- Programa
DROP TRIGGER IF EXISTS audit_programa_trigger ON program_programa;
CREATE TRIGGER audit_programa_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_programa
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Regras do Programa
DROP TRIGGER IF EXISTS audit_regraprograma_trigger ON program_regraprograma;
CREATE TRIGGER audit_regraprograma_trigger
```

```

AFTER INSERT OR UPDATE OR DELETE ON program_regraprograma
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Imóveis
DROP TRIGGER IF EXISTS audit_imovel_trigger ON program_imovel;
CREATE TRIGGER audit_imovel_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_imovel
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Vínculo N:N entre Imóveis e Moradores
DROP TRIGGER IF EXISTS audit_imovel_moradores_trigger ON program_imovel_moradores;
CREATE TRIGGER audit_imovel_moradores_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_imovel_moradores
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Saldo de Pontos
DROP TRIGGER IF EXISTS audit_saldopontos_trigger ON program_saldopontos;
CREATE TRIGGER audit_saldopontos_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_saldopontos
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Consolidação
DROP TRIGGER IF EXISTS audit_consolidacao_trigger ON program_consolidacao;
CREATE TRIGGER audit_consolidacao_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_consolidacao
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Constante de Pontuação
DROP TRIGGER IF EXISTS audit_constantepontuacao_trigger ON program_constantepontuacao;
CREATE TRIGGER audit_constantepontuacao_trigger
AFTER INSERT OR UPDATE OR DELETE ON program_constantepontua

```

```
cao
```

```
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```

App: collection

```
-- Registro da Coleta
DROP TRIGGER IF EXISTS audit_registrocoleta_trigger ON collection_registrocoleta;
CREATE TRIGGER audit_registrocoleta_trigger
AFTER INSERT OR UPDATE OR DELETE ON collection_registrocoleta
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Evidência
DROP TRIGGER IF EXISTS audit_evidencia_trigger ON collection_evidencia;
CREATE TRIGGER audit_evidencia_trigger
AFTER INSERT OR UPDATE OR DELETE ON collection_evidencia
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();

-- Contestação
DROP TRIGGER IF EXISTS audit_contestacao_trigger ON collection_contestacao;
CREATE TRIGGER audit_contestacao_trigger
AFTER INSERT OR UPDATE OR DELETE ON collection_contestacao
FOR EACH ROW EXECUTE FUNCTION audit.audit_trigger_func();
```



3. Roteiro de Testes Homologação

Utilize os blocos de comando ordenados abaixo para validar se os gatilhos estão interceptando as transações corretamente.

Etapa 1: Validar Inserção (**INSERT**)

```
-- Insere um novo Role
INSERT INTO accounts_role (nome, descricao, ativo)
VALUES ('Auditor_Teste', 'Papel criado para testar os trigg
```

```
ers de auditoria', true);

-- Verifica o registro na tabela nativa
SELECT * FROM accounts_role WHERE nome = 'Auditor_Testes';
```

Etapa 2: Validar Alteração (UPDATE)

```
-- Atualiza os dados do Role recém-criado
UPDATE accounts_role
SET descricao = 'Descrição alterada após o UPDATE', ativo =
false
WHERE nome = 'Auditor_Testes';

-- Verifica as alterações aplicadas
SELECT * FROM accounts_role WHERE nome = 'Auditor_Testes';
```

Etapa 3: Validar Remoção (DELETE)

```
-- Deleta o Role do banco
DELETE FROM accounts_role WHERE nome = 'Auditor_Testes';

-- Valida se a tabela original está limpa (deve retornar 0
linhas)
SELECT * FROM accounts_role WHERE nome = 'Auditor_Testes';
```

Etapa 4: Auditoria dos Resultados

```
-- Busca as últimas transações registradas pela esteira de
auditoria
SELECT
    id,
    table_name,
    action_type,
    record_id,
    old_data,
    new_data,
    changed_at,
```

```
db_user
FROM audit.sistema_audit_log
ORDER BY changed_at DESC
LIMIT 5;
```

Monitoramento com grafana:

Nessa situação vamos criar um container para o grafana, no arquivo `docker-compose.yml` vamos adicionar dentro do

```
services:
# ....
# Grafana
grafana:
  image: grafana/grafana:latest
  container_name: coleta-premiada-grafana
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=senha_admin_grafana
  volumes:
    - grafana-data:/var/lib/grafana
  depends_on:
    - db # Substitua 'db' pelo nome real do serviço do postgres no seu arquivo (ex: postgres)
# .....
```

obs: (Não se esqueça de adicionar grafana-data: na declaração de volumes no final do seu arquivo docker-compose).

Depois, suba o container rodando no terminal:

```
docker compose up -d grafana
```

1. Acesse o Grafana no navegador: <http://localhost:3000>
2. Faça login com o usuário admin e a senha senha_admin_grafana.

3. No menu lateral esquerdo, vá em Connections (ícone de tomada) > Data sources.
4. Clique em Add data source e escolha PostgreSQL.
5. Preencha os dados:
 - Host: `coleta-premiada-postgres-1:5432` (O Grafana usa o nome do container para se comunicar na rede interna do Docker).
 - Database: `coleta_pg`
 - User: `coleta_user`
 - Password: `123`
 - TLS/SSL Mode: Coloque como disable (está rodando local).
6. Role até o fim e clique em Save & Test. Deve aparecer uma mensagem verde de sucesso.
7. Criação de alguns Painéis de monitoramento

New tab

▼ New row

Auditoria_geral

	id	table_name	action_type	record_id	old_data	new_data	changed_at	db_user
	1	accounts_usuario	INSERT		2	{ "id": 2, "cpf": "99999999"	2026-05-13 20:33:18	coleta_user
	2	accounts_usuario	UPDATE		2 { "id": 2, "cpf": "99999999"	{ "id": 2, "cpf": "99999999"	2026-05-13 20:38:23	coleta_user
	3	accounts_usuario	DELETE		2 { "id": 2, "cpf": "99999999"		2026-05-13 20:40:58	coleta_user
	4	accounts_usuario	INSERT		3	{ "id": 3, "cpf": "315.804.9"	2026-05-15 13:43:29	coleta_user
	5	program_imovel	INSERT		1	{ "id": 1, "cep": "55555555"	2026-05-15 13:44:08	coleta_user
	6	collection_registrocoleta	INSERT		1	{ "id": 1, "imovelId": 1, "pr	2026-05-15 13:51:50	coleta_user

▼ New row 1

Tabela de Alterações de Cadastro

Data	Ação	Quem Alterou	Nome Antigo	Nome Novo	Email
2026-05-13 20:40:58	DELETE	coleta_user	Morador Editado (Auditoria)		morador.teste.auditoria@email.com
2026-05-13 20:38:23	UPDATE	coleta_user	Morador de Teste Auditoria	Morador Editado (Auditoria)	morador.teste.auditoria@email.com

